

Project - High Level Design

On

IaC Provisioning for Agriculture

Support Triage Agent System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	DEEPAK PATRO	EN22CS301316
2.	BHOOMIKA PATIDAR	EN22CS301270
3.	CHARU NARIYA	EN22CS301290
4.	DEV KUMAR DAS	EN22CS301321
5.	AVICHAL MISHRA	EN22CS301243

Group Name: 07D3

Project Number: AAI-31

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

Table of Contents

1.Introduction

The Smart Agriculture AI Assistant is a cloud-based application developed using DevOps practices to provide intelligent farming assistance. The system uses AI to answer queries related to crops, diseases, and agricultural policies. The project integrates development, deployment, and infrastructure automation into a single pipeline using modern tools.

Key Points:

- AI-powered agriculture assistance system
- Cloud-hosted application on AWS EC2
- Fully automated deployment using DevOps pipeline
- Reduces manual effort and configuration errors
- Provides real-time responses to users

1.1 Scope of the Document

This document defines the overall architecture and high-level working of the system. It includes application design, infrastructure setup, CI/CD pipeline, and system interactions.

Scope Includes:

- System architecture overview
- DevOps pipeline design
- Infrastructure provisioning using Terraform (main.tf)
- Containerization using Docker (Dockerfile)
- Deployment automation using GitHub Actions (deploy.yml)

1.2 Intended Audience

This document is intended for:

Audience:

- Project evaluators and faculty
- DevOps engineers and developers
- Students and project team members
- Anyone interested in understanding system architecture

1.3 System Overview

The system is a web-based AI assistant where users interact through a browser interface. The backend processes user queries using AI APIs and returns intelligent responses.

System Features:

- User-friendly web interface (Streamlit)
- AI-based query processing
- Cloud deployment on AWS
- Automated CI/CD pipeline
- Secure handling of API keys

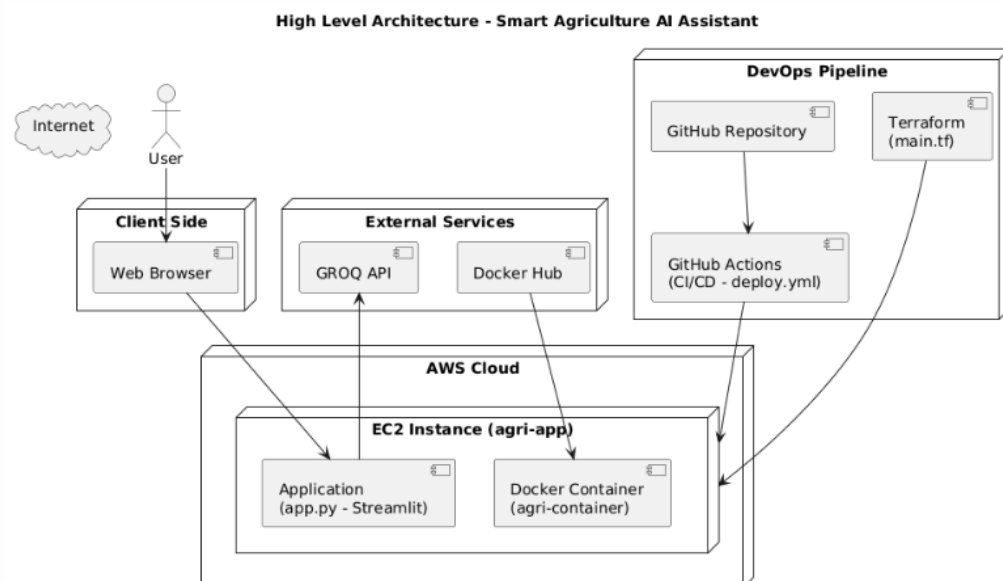
2. System Design

2.1 Application Design

The application is developed using Streamlit in app.py, which provides an interactive UI. The backend integrates with AI APIs for generating responses.

Design Points:

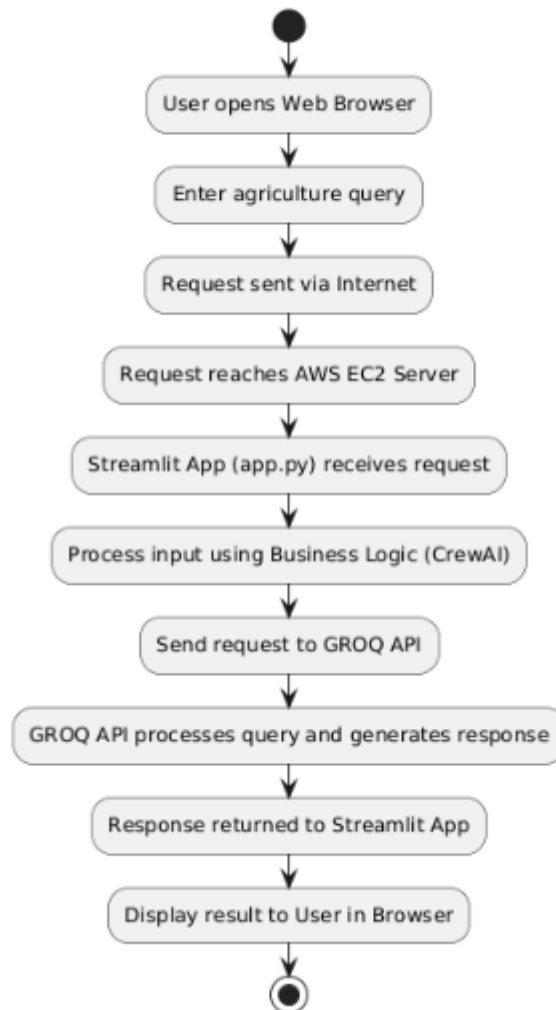
- Frontend + backend combined in app.py
- Uses Streamlit for UI rendering
- AI processing via GROQ API
- Lightweight and fast execution
- Runs on port 8501



2.2 Process Flow

1. User opens application in browser
2. User enters query (e.g., crop issue)
3. Application processes input (app.py)
4. Request sent to GROQ API
5. AI generates response
6. Response displayed on UI

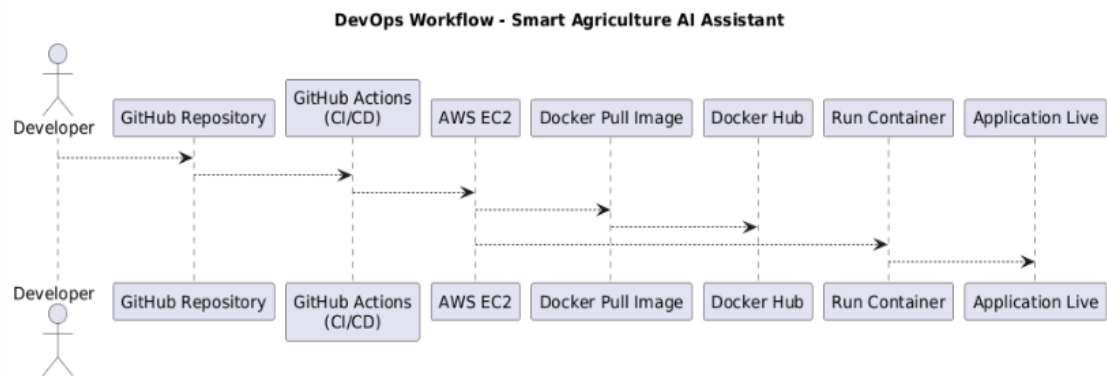
Smart Agriculture AI Assistant - Process Flow



2.3 Information Flow

Data Flow:

- User → Web Interface
- Web Interface → Application (app.py)
- Application → GROQ API
- API → Application
- Application → User



2.4 Components Design

1. Frontend

- Built using Streamlit
- File: app.py
- Handles user interaction

2. Backend

- AI logic using CrewAI
- Processes queries and API calls

3. Containerization

- Docker used for packaging
- File: Dockerfile
- Ensures consistent environment

4. CI/CD Pipeline

- GitHub Actions
- File: .github/workflows/deploy.yml
- Automates deployment

5. Infrastructure

- Terraform
- File: main.tf
- Creates EC2 instance

6. Cloud Server

- AWS EC2 (agri-app)
- Runs Docker container

2.5 Key Design Considerations

- **Automation:** Fully automated deployment
- **Scalability:** Can handle multiple users
- **Security:** API keys stored in GitHub Secrets
- **Reliability:** CI/CD ensures consistent deployment
- **Portability:** Docker allows deployment anywhere

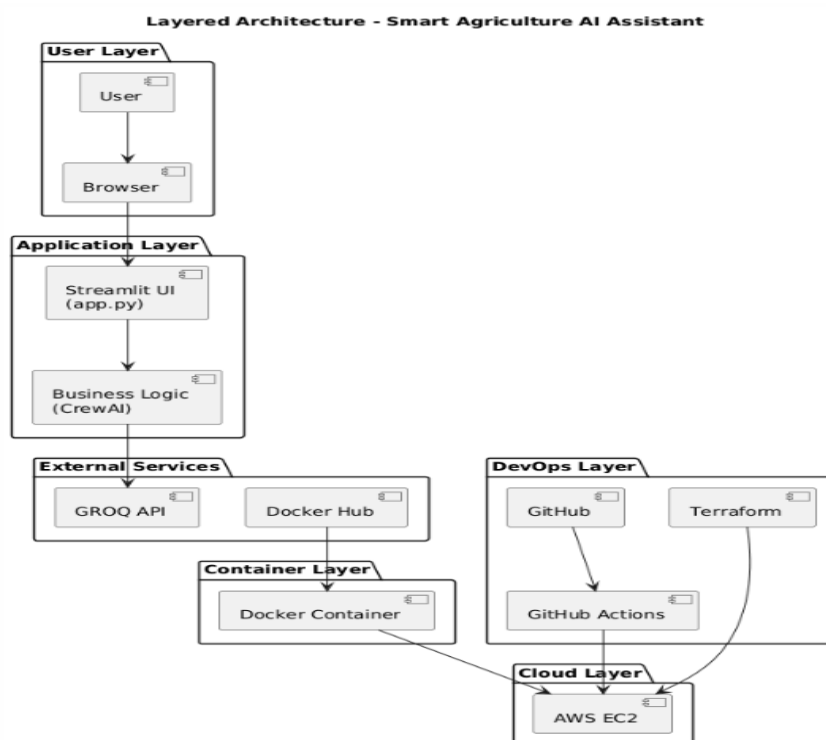
- **Maintainability:** Code is version-controlled

2.6 API Catalogue

API	Description
GROQ API	Used for AI response generation

API Features:

- Real-time response generation
- Handles natural language queries
- Secure access using API key



3. Data Design

3.1 Data Model

Key data entities include:

- User Queries
- AI Responses
- Knowledge Base Documents
- Vector Embeddings

3.2 Data Access Mechanism

- Backend accesses vector DB via RAG pipeline
- APIs used for communication
- LLM accessed through external API

- Temporary storage of user queries
- Logs retained for monitoring
- Long-term storage for knowledge base

3.4 Data Migration

- Terraform ensures reproducible infrastructure
- Docker ensures consistent environments
- Easy migration across environments

4. Interfaces

- **User Interface:** Streamlit web app
- **API Interface:** GROQ API
- **Deployment Interface:** SSH to EC2
- **CI/CD Interface:** GitHub Actions

5. State and Session Management

- Managed by Streamlit
- Temporary session storage
- No persistent session tracking

6. Caching

- Basic caching via Streamlit
- Improves performance
- Reduces repeated computation

7. Non-Functional Requirements**7.1 Security Aspects****Security Measures:**

- API keys stored in GitHub Secrets
- Secure SSH connection
- No sensitive data in code
- Controlled access to EC2

7.2 Performance Aspects

- Fast deployment using Docker
- Efficient CI/CD pipeline
- Cloud-based scalability
- Quick response via AI API

8. References

- Docker Official Documentation
- Terraform Documentation
- GitHub Actions Documentation
- GROQ API Documentation